Backtracking & Branch-and-bound

Backtracking

Hard problems

- OK, someone's brought us a problem to solve.
- We couldn't find a polynomial time algorithm…but we managed to show it's NP-complete
- So, job done?
- Well, maybe we still want to solve it as fast as we can

Hard problems

- Let's say the problem is CNF-SAT
- How should I go about solving this?
- Idea 1: exhaustively try all assignments of the variables.
- So, for each function a: $\{x_1, \ldots, x_n\} \to \{0,1\}$, check whether a makes the formula true.
- How long will this take?
- Can we do better?

The writing on the wall

- Let's say our formula contains the clause $(x_1 + \neg x_2)$.
- Then any assignment which sets x_1 to 0 and x_2 to 1 is guaranteed to make the formula false.
- So it's a bit stupid to waste time checking all 2^{n-2} of those assignments!
- Idea: let's build up our assignments variable by variable, checking as we go whether we're already dead in which case we can give up on this path (`*backtrack*')

Backtracking

- We'll maintain a set of *partial* assignments, where each variable can be set to 0, set to 1 or not yet set. Let a_0 be the assignment with nothing set.
- For a partial assignment a, in which the variable i is **not** yet set, define the set `branch(a,i)` as the two partial assignments that result from setting variable i to 1 or 0.
- Define `contr(a,ϕ)` to be true if a already falsifies a clause of φ and false otherwise (i.e. "are we dead?").
- Define `complete(a)` to be true if a has all the variables set.

Bactracking

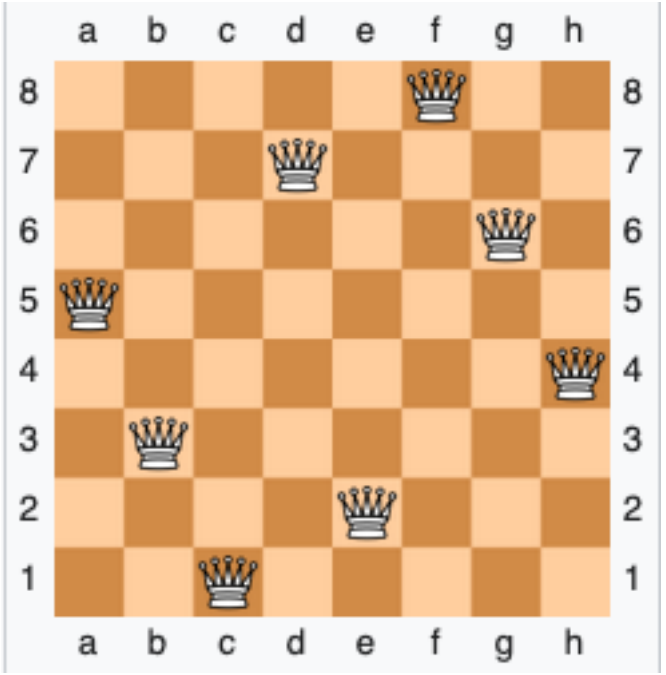
```
solve(ϕ):
  set X = {a_0}
  while X is nonempty:
    pick a in X and i such that variable i is not
set in X
    for b in branch(a,i):
      if contr(b,ϕ): continue
      if complete(b): return b
      X.add(b)
  return UNSAT
```

Backtracking

- What can we say about this algorithm?
- Correctness? Runtime?
- For correctness, key fact: any solution extending a must extend one of the elements of branch(a,i).
- Let’s try it out…

Testbed: n-queens problem

- We need a family of hard-ish formulae to test our algorithm on
- The: *n-queens puzzle*
- How do I encode this into a formula?



Backtracking, generically

- Define a notion of *partial solution*.
- Define a way of extending partial solutions, extend, giving a set of partial solutions extend(a,i) for every partial solution a and (optional) parameter i.
- Important property: for every choice of i, every partial solution extending a extends an element of extend(a,i).
- Define a function dead to test for partial solutions that can’t be extended, and a function complete to test for being a complete solution.

Backtracking, generically

```
solve:
  set X = {a_0}
  while X is nonempty:
    pick a in X and i allowed for a
    for b in extend(a,i):
      if dead(b): continue
      if complete(b): return b
      X.add(b)
  return NO
```

Example: vertex cover

- Write a backtracking algorithm to solve the vertex-cover problem (what is the decision version?)
- What should the set of partial solutions be?
- What should the functions dead and complete be?
- What should the extend functions be?
- Now write down pseudocode for the algorithm.

Branch-and-bound

Branch-and-bound

- Let’s think about optimisation problems
- One thing we can do is convert to the decision version and then binary search.
- But that’s a bit silly, we should just search for the best we can find in one go.
- So when can we give up on a partial solution?
- When it’s already worse than one we know.
- Or when we know it must end up worse than one we know.

Branch-and-bound

- Assume a function cost for the cost of full solutions.
- So we need a function bound(a) giving a lower bound on the cost (/upper bound on the value) of any solution extending a.

Branch-and-bound

```
solve:
  set X = {a_0}
  set bestcost = INFTY
  while X is nonempty:
    pick a in X and i allowed for a
    for b in extend(a,i):
      if bound(b) > bestcost: continue
      if complete(b):
        if cost(b) < bestcost:
          bestcost = cost(b)
          bestsol = b
      else: X.add(b)
  return bestcost, bestsol
```

Branch-and-bound

- So where do we get the function bound?
- Could use something simple (e.g. just the cost so far)
- What could we do for, say, knapsack?
- For vertex cover: greedily pick a set of edges that don't share any endpoints – we must use at least that many more vertices.
- What about generically?
- Next lecture we'll see something we can do.

Exercises

- Programme a backtracking algorithm to solve sudoku puzzles.
- GT C-18.2, C-18.3, C-18.4